

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет «МИЭТ»

Институт системной и программной инженерии и информационных технологий

Дисциплина: Основы программной инженерии

Отчет по лабораторной работе №3

«Интеграционное тестирование»

Вариант 3

Выполнил:

студент группы П-42

Варлапов Артем

Москва, 2026

Цель работы

Научиться выполнять интеграционное тестирование ПО.

Задание

Сформировать иерархию классов вокруг ранее протестированного класса. Провести модульное тестирование написанных классов и инкрементальное интеграционное тестирование сверху вниз с использованием программных заглушек для имитации ещё не подключённых классов нижнего уровня.

Для варианта 3 исходным является класс «Строка» `TextLine` из лабораторной работы №2. В лабораторной работе №3 отдельного списка вариантов нет, поэтому ссылка на класс из лабораторной №3 трактуется как ссылка на ранее протестированный класс из лабораторной №2.

Выполнение работы

1 Иерархия и взаимодействие классов

`TextLine` является базовым классом строки. Классы `RussianTextLine` и `EnglishTextLine` наследуют `TextLine` и задают язык при создании объекта. `TextDocument` является агрегирующим классом: он хранит список объектов `TextLine` и суммирует их текущие длины.

Связи классов: `RussianTextLine` → `TextLine` и `EnglishTextLine` → `TextLine` являются наследованием; `TextDocument` → список `TextLine` является агрегацией. При вычислении длины документа вызывается `getLength()` каждого объекта строки.

Класс	Назначение
<code>TextLine</code>	Базовый класс строки: хранит массив символов, длину и язык; поддерживает изменение длины и конкатенацию.
<code>RussianTextLine</code>	Наследник <code>TextLine</code> , задающий язык «Русский» при создании объекта.
<code>EnglishTextLine</code>	Наследник <code>TextLine</code> , задающий язык «Английский» при создании объекта.
<code>TextDocument</code>	Агрегирующий класс, хранящий список строк и вычисляющий суммарную длину документа.
<code>TextLineStub</code>	Программная заглушка для верхнеуровневого тестирования <code>TextDocument</code> .

2 Реализация классов

Базовый класс `TextLine` перенесён из лабораторной работы №2. Некорректные аргументы обрабатываются через `IllegalArgumentException`. Наследники задают фиксированный язык, а `TextDocument` добавляет проверку `null` при добавлении строки.

TextLine.java

```
import java.util.Arrays;

public class TextLine {
    private int length;
    private char[] characters;
    private String language;

    public TextLine(String text, String language) {
        if (text == null) {
            throw new IllegalArgumentException("Text must not be null");
        }
        setLanguage(language);
        characters = text.toCharArray();
        length = characters.length;
    }

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        if (length < 0) {
            throw new IllegalArgumentException("Length must not be negative");
        }
        characters = Arrays.copyOf(characters, length);
        this.length = characters.length;
    }

    public String getLanguage() {
        return language;
    }

    public void setLanguage(String language) {
        if (language == null || language.trim().isEmpty()) {
            throw new IllegalArgumentException("Language must not be empty");
        }
        this.language = language;
    }
}
```

```

        throw new IllegalArgumentException("Language must not be empty");
    }
    this.language = language;
}

public void append(TextLine other) {
    if (other == null) {
        throw new IllegalArgumentException("Other line must not be null");
    }
    char[] result = Arrays.copyOf(characters, length + other.length);
    System.arraycopy(other.characters, 0, result, length, other.length);
    characters = result;
    length = characters.length;
}
}

```

RussianTextLine.java

```

public class RussianTextLine extends TextLine {
    public RussianTextLine(String text) {
        super(text, "Русский");
    }
}

```

EnglishTextLine.java

```

public class EnglishTextLine extends TextLine {
    public EnglishTextLine(String text) {
        super(text, "Английский");
    }
}

```

TextDocument.java

```
import java.util.ArrayList;
import java.util.List;

public class TextDocument {
    private final List<TextLine> lines = new ArrayList<>();

    public void addLine(TextLine line) {
        if (line == null) {
            throw new IllegalArgumentException("Line must not be null");
        }
        lines.add(line);
    }

    public int getLength() {
        int length = 0;
        for (TextLine line : lines) {
            length += line.getLength();
        }
        return length;
    }
}
```

3 Заглушка и тестирование сверху вниз

На первом этапе проверяется верхний класс TextDocument. Настоящие строки заменяются объектами TextLineStub. Переопределённый метод getLength() возвращает заданное тестом число независимо от внутреннего массива символов. Наследование используется для совместимости с интерфейсом TextLine.

TextLineStub.java

```
public class TextLineStub extends TextLine {
    private final int fixedLength;

    public TextLineStub(int fixedLength) {
        super("", "Заглушка");
        this.fixedLength = fixedLength;
    }

    @Override
    public int getLength() {
        return fixedLength;
    }
}
```

Этап 1: TextDocumentTest проверяет пустой документ, одну заглушку, сумму длин нескольких заглушек и отклонение null. Так проверяется логика документа до подключения настоящих строк.

Этап 2: в тесте documentWorksWithRealTextLines заглушки заменяются настоящими объектами TextLine. Проверяется сумма длин строк «abc» и «de», равная 5 символам.

Этап 3: в тестах documentWorksWithBothSubclasses и documentReflectsChangesInItsLines подключаются наследники. Проверяется сумма длин русской и английской строк, а также изменение длины документа после append() и setLength() у сохранённой строки.

4 Модульные тесты

Написаны тесты всех четырёх классов программы. Всего подготовлено 24 модульных теста. Проверяются корректные данные, пустые строки, граничные длины и ошибочные аргументы.

Тестовый класс	Количество тестов	Что проверяется
TextLineTest	14	Проверка конструктора, изменения языка, длины, конкатенации и ошибочных аргументов.
RussianTextLineTest	3	Проверка языка, длины пустой строки и обработки null.
EnglishTextLineTest	3	Проверка языка, длины пустой строки и обработки null.
TextDocumentTest	4	Верхнеуровневая проверка документа с программными заглушками.
TextDocumentIntegrationTest	3	Проверка взаимодействия документа с настоящими строками и наследниками.

TextLineTest.java

```
import java.lang.reflect.Field;
import org.junit.Test;

import static org.junit.Assert.*;

public class TextLineTest {
    private char[] charactersOf(TextLine line) throws Exception {
        Field field = TextLine.class.getDeclaredField("characters");
        field.setAccessible(true);
        return (char[]) field.get(line);
    }
}
```

```

@Test
public void constructorStoresFields() throws Exception {
    TextLine line = new TextLine("Привет", "Русский");
    assertEquals(6, line.getLength());
    assertEquals("Русский", line.getLanguage());
    assertEquals("Привет".toCharArray(), charactersOf(line));
}

```

```

@Test
public void emptyTextHasZeroLength() throws Exception {
    TextLine line = new TextLine("", "Русский");
    assertEquals(0, line.getLength());
    assertEquals(new char[0], charactersOf(line));
}

```

```

@Test
public void languageCanBeChanged() {
    TextLine line = new TextLine("abc", "Русский");
    line.setLanguage("Английский");
    assertEquals("Английский", line.getLanguage());
    assertEquals(3, line.getLength());
}

```

```

@Test
public void lengthCanBeReducedIncreasedAndSetToZero() throws Exception {
    TextLine line = new TextLine("abc", "Английский");
    line.setLength(2);
    assertEquals(2, line.getLength());
    assertEquals(new char[] { 'a', 'b' }, charactersOf(line));
    line.setLength(4);
    assertEquals(4, line.getLength());
    assertEquals(new char[] { 'a', 'b', '\0', '\0' }, charactersOf(line));
    line.setLength(0);
    assertEquals(0, line.getLength());
    assertEquals(new char[0], charactersOf(line));
}

```

```
}
```

```
@Test
```

```
public void appendCopiesCharactersAndRecalculatesLength() throws  
Exception {  
    TextLine first = new TextLine("Привет, ", "Русский");  
    TextLine second = new TextLine("world", "Английский");  
    first.append(second);  
    assertEquals(13, first.getLength());  
    assertEquals("Привет, world".toCharArray(), charactersOf(first));  
    assertEquals("Русский", first.getLanguage());  
    assertEquals(5, second.getLength());  
    assertEquals("world".toCharArray(), charactersOf(second));  
}
```

```
@Test
```

```
public void appendHandlesEmptyLines() throws Exception {  
    TextLine line = new TextLine("", "Английский");  
    line.append(new TextLine("abc", "Английский"));  
    line.append(new TextLine("", "Английский"));  
    assertEquals(3, line.getLength());  
    assertEquals("abc".toCharArray(), charactersOf(line));  
}
```

```
@Test
```

```
public void lineCanBeAppendedToItself() throws Exception {  
    TextLine line = new TextLine("ab", "Английский");  
    line.append(line);  
    assertEquals(4, line.getLength());  
    assertEquals("abab".toCharArray(), charactersOf(line));  
}
```

```
@Test(expected = IllegalArgumentException.class)
```

```
public void constructorRejectsNullText() {  
    new TextLine(null, "Русский");  
}
```

```

@Test(expected = IllegalArgumentException.class)
public void constructorRejectsNullLanguage() {
    new TextLine("abc", null);
}

@Test(expected = IllegalArgumentException.class)
public void constructorRejectsEmptyLanguage() {
    new TextLine("abc", "");
}

@Test(expected = IllegalArgumentException.class)
public void setterRejectsBlankLanguage() {
    new TextLine("abc", "Русский").setLanguage(" ");
}

@Test(expected = IllegalArgumentException.class)
public void setterRejectsNullLanguage() {
    new TextLine("abc", "Русский").setLanguage(null);
}

@Test(expected = IllegalArgumentException.class)
public void lengthCannotBeNegative() {
    new TextLine("abc", "Английский").setLength(-1);
}

@Test(expected = IllegalArgumentException.class)
public void appendRejectsNull() {
    new TextLine("abc", "Английский").append(null);
}
}

```

RussianTextLineTest.java

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class RussianTextLineTest {
    @Test
    public void constructorSetsLanguageAndLength() {
        RussianTextLine line = new RussianTextLine("Привет");
        assertEquals("Русский", line.getLanguage());
        assertEquals(6, line.getLength());
    }

    @Test
    public void emptyLineHasZeroLength() {
        assertEquals(0, new RussianTextLine("").getLength());
    }

    @Test(expected = IllegalArgumentException.class)
    public void constructorRejectsNullText() {
        new RussianTextLine(null);
    }
}
```

EnglishTextLineTest.java

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class EnglishTextLineTest {
    @Test
    public void constructorSetsLanguageAndLength() {
        EnglishTextLine line = new EnglishTextLine("hello");
        assertEquals("Английский", line.getLanguage());
        assertEquals(5, line.getLength());
    }
}
```

```

    }

    @Test
    public void emptyLineHasZeroLength() {
        assertEquals(0, new EnglishTextLine("").getLength());
    }

    @Test(expected = IllegalArgumentException.class)
    public void constructorRejectsNullText() {
        new EnglishTextLine(null);
    }
}

```

TextDocumentTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TextDocumentTest {

    @Test
    public void emptyDocumentHasZeroLength() {
        assertEquals(0, new TextDocument().getLength());
    }

    @Test
    public void documentUsesLengthProvidedByStub() {
        TextDocument document = new TextDocument();
        document.addLine(new TextLineStub(7));
        assertEquals(7, document.getLength());
    }

    @Test
    public void documentSumsStubLengths() {
        TextDocument document = new TextDocument();
        document.addLine(new TextLineStub(3));
    }
}

```

```

        document.addLine(new TextLineStub(0));
        document.addLine(new TextLineStub(5));
        assertEquals(8, document.getLength());
    }

    @Test(expected = IllegalArgumentException.class)
    public void documentRejectsNullLine() {
        new TextDocument().addLine(null);
    }
}

```

5 Интеграционные тесты

Три интеграционных теста проверяют взаимодействие документа с настоящим базовым классом и наследниками, включая пересчёт общей длины после изменения строки.

TextDocumentIntegrationTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TextDocumentIntegrationTest {
    @Test
    public void documentWorksWithRealTextLines() {
        TextDocument document = new TextDocument();
        document.addLine(new TextLine("abc", "Английский"));
        document.addLine(new TextLine("de", "Английский"));
        assertEquals(5, document.getLength());
    }

    // Этап 3: подключены классы-наследники.
    @Test
    public void documentWorksWithBothSubclasses() {
        TextDocument document = new TextDocument();
        document.addLine(new RussianTextLine("Привет"));
        document.addLine(new EnglishTextLine("hello"));
    }
}

```

```

        assertEquals(11, document.getLength());
    }

    @Test
    public void documentReflectsChangesInItsLines() {
        TextDocument document = new TextDocument();
        RussianTextLine line = new RussianTextLine("Привет");
        document.addLine(line);
        assertEquals(6, document.getLength());
        line.append(new RussianTextLine("!"));
        assertEquals(7, document.getLength());
        line.setLength(2);
        assertEquals(2, document.getLength());
    }
}

```

6 Результаты тестирования

Сначала отдельно запущена проверка верхнего класса с заглушками. Все 4 теста прошли успешно.

Результат проверки `TextDocument` с программными заглушками

```
JUnit version 4.13.2
```

```
....
```

```
Time: 0.008
```

```
OK (4 tests)
```

Затем запущен полный набор модульных и интеграционных тестов через JUnitCore из JUnit 4.13.2. Все 27 тестов прошли успешно.

Результат выполнения полного набора тестов

```
JUnit version 4.13.2
```

```
.....
```

```
Time: 0.015
```

```
OK (27 tests)
```

Вывод

Удалось научиться выполнять интеграционное тестирование ПО